Case Study Report

on

Containerized Deployment of Node.js Applications using Docker on AWS EC2 Instance

Bachelor of Technology

in

Computer Science Engineering (Artificial Intelligence & Machine Learning)

By

Student Name	Roll No
Basava Lokesh	A23126552134
Ganta Cheradeep	A23126552143
Gopidalai Akhil	A23126552147
Tatisetti Teja Kumar	A23126552177
Yaka Anil	A23126553188

III/IV B.Tech, II-Semester

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Sangivalasa, Bheemunipatnam Mandal,

Visakhapatnam District, AP, India

2026

CERTIFICATE

The case study report entitled "**Containerized Deployment of Node.js Applications using Docker on AWS EC2 Instance**" submitted by Basava Lokesh , Ganta Cheradeep , Gopidalai Akhil , Tatisetti Teja Kumar , Yaka Anil III/IV B.Tech, II-Semester bearing Regd. No. A23126552134, A23126552143 , A23126552147 , A23126552177 , A23126552188 Dept. of CSE(AI&ML) is found to be satisfactory.

Mr. Santosh Kumar P
Associate Professor
Dept. of CSE(AI&ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world cloud and containerization environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering at ANITS, and our guide Mr. Santosh Kumar P, Associate Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support.

Regd. No. A23126552134
A23126552143
A23126552147
A23126552177
A23126552188

CONTENTS

Sl. No.	Title	Page No.
1	Abstract	5
2	Introduction	5
3	Aim	6
4	Objective	6
5	Tools Used	7
6	Architecture Diagram	8
7	Procedure	9
8	Implementation	9
9	Result	12
10	Conclusion	14
11	Advantages	14
12	Limitations	15
13	References	15

Abstract

This case study demonstrates the end-to-end deployment of a Node.js Express web application using Docker on an Amazon Web Services (AWS) EC2 instance running Amazon Linux 2023. The entire development and deployment workflow is performed remotely on the EC2 instance without requiring any local development environment. Docker is used to containerize the application, providing a consistent and isolated runtime environment. Nodemon is integrated for live reload functionality, enabling real-time code changes to reflect instantly inside the running container through Docker volume mounting. The application is built using Express.js and exposed on port 3000, accessible via the public IP of the EC2 instance. This study highlights the practical use of cloud computing, containerization, and DevOps practices in modern web application development.

Introduction

Cloud computing and containerization have fundamentally transformed the way modern web applications are developed, deployed, and managed. Amazon Web Services (AWS) Elastic Compute Cloud (EC2) provides on-demand virtual servers that developers can use as fully managed remote development and hosting environments, eliminating the need for physical hardware or local setup.

Docker is a leading containerization platform that packages applications and their dependencies into isolated containers, ensuring that the application behaves consistently across all environments. When combined with AWS EC2, Docker enables cloud-native development workflows that are highly reproducible, scalable, and portable.

Node.js is a powerful JavaScript runtime environment widely used for building fast and scalable server-side applications. Express.js, a minimal web framework for Node.js, simplifies HTTP server creation and routing. Nodemon, a development utility, automatically restarts the Node.js server when file changes are detected, enabling live reload functionality critical for rapid development.

This case study integrates all these technologies to build a Docker-based Node.js development environment entirely within an AWS EC2 instance, demonstrating a practical and professional cloud development workflow.

Aim

To deploy a Node.js (Express) web application inside a Docker container on an AWS EC2 instance running Amazon Linux 2023, and to demonstrate live reload development capability using Nodemon — all without requiring any local development environment on the developer's machine.

Objective

The primary objectives of this case study are:

1. To set up and configure an AWS EC2 instance (Amazon Linux 2023) as a remote cloud development machine accessible via SSH.
2. To install and configure Node.js, npm, and Docker directly on the EC2 instance without any local dependencies.
3. To create a Node.js Express web application entirely within the EC2 environment using the terminal text editor nano.
4. To containerize the Node.js application using Docker with a custom Dockerfile based on the official node:18 base image.
5. To enable live reload using Nodemon through Docker volume mounting, so that code changes reflect instantly without restarting the container.
6. To expose the application on port 3000 and access it from a browser using the EC2 public IP address after configuring the AWS Security Group.
7. To verify the complete workflow by demonstrating both initial application output and live reload functionality using browser screenshots.

Tools Used

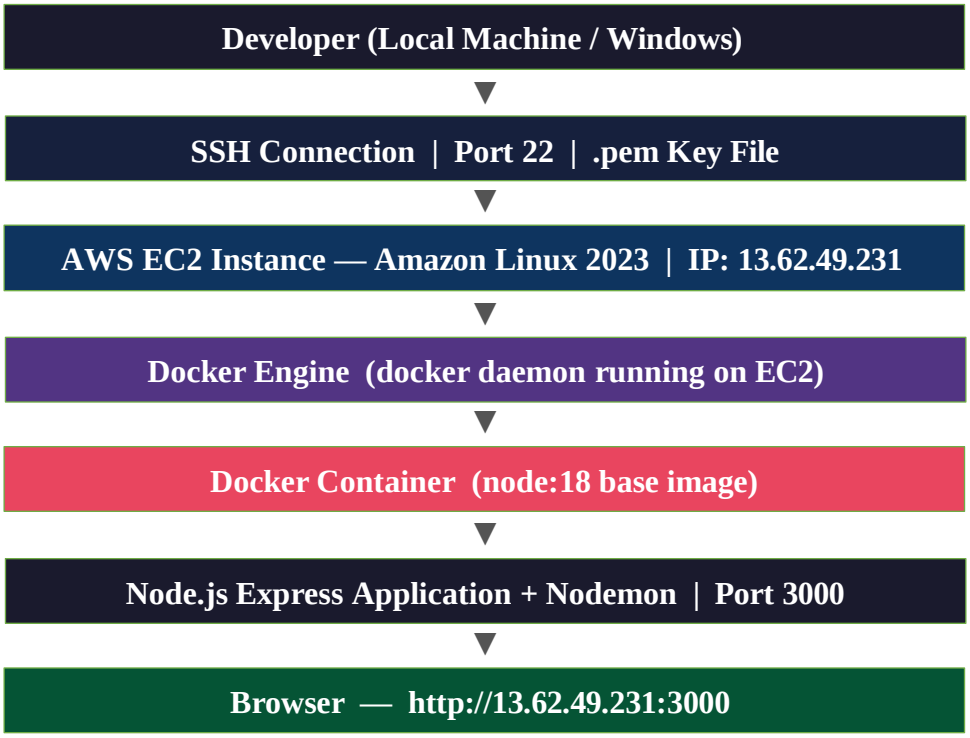
The following tools and technologies were used in this case study:

Tool / Technology	Version / Details	Purpose
AWS EC2	`Amazon Linux 2023`	Cloud virtual machine for remote development and hosting
Node.js	`v18.20.8`	JavaScript runtime for building the backend application
npm	`v10.8.2`	Package manager to install Node.js dependencies
Express.js	`v5.2.1`	Minimal web framework for HTTP server and routing
Nodemon	`v3.1.14`	Monitors file changes and auto-restarts the server
Docker	`v25.0.14`	Containerization platform to isolate and run the app
node:18 Image	`Docker Hub`	Official base image for building the container
nano	`Built-in (Linux)`	Terminal text editor used to write all project files
SSH / PuTTY	`Windows Client`	Secure shell access to connect to EC2 remotely
AWS Security Group	`AWS Console`	Firewall configuration to allow inbound port 3000 traffic

Architecture Diagram

The following diagram illustrates the complete system architecture — from the developer's local machine through to the running containerized application accessible from a browser:

Figure 1: System Architecture Flow



Volume Mount: EC2 ~/node-docker-app/ ↔ Container /app/ (Live Reload via Nodemon)

Procedure

The following high-level steps were followed to complete this case study:

8. Connect to the AWS EC2 instance via SSH using a .pem key file from a Windows machine.
9. Update the system package index and install Node.js v18 and npm using the dnf package manager.
10. Install Docker from the Amazon Linux repository, start the Docker service, and enable it on boot.
11. Add the ec2-user to the docker group to allow running Docker commands without sudo.
12. Exit the SSH session and reconnect so the group permission changes take effect.
13. Create a project directory (node-docker-app) and initialize a Node.js project using npm init.
14. Install Express.js as a production dependency and Nodemon as a development dependency.
15. Create the main application file (app.js) and update package.json with start and dev scripts.
16. Write a Dockerfile specifying node:18 as the base image with the appropriate build instructions.
17. Create a .dockerignore file to exclude node_modules and log files from the Docker build context.
18. Build the Docker image using the docker build command.
19. Run the Docker container with port mapping (-p 3000:3000) and volume mounting (-v) for live reload.
20. Configure the EC2 Security Group to allow inbound TCP traffic on port 3000.
21. Access the application from a browser, verify the output, test live reload by editing app.js.

Implementation

Step 1: Connect to EC2

From a Windows machine, connect to the EC2 instance using SSH with the .pem private key file:

```
ssh -i internal.pem ec2-user@13.62.49.231
```

Step 2: Install Node.js

Update the Amazon Linux 2023 package index and install Node.js 18:

```
sudo dnf update -y
sudo dnf install nodejs -y
node -v    # Output: v18.20.8
npm -v     # Output: 10.8.2
```

Step 3: Install and Configure Docker

Install Docker from the Amazon Linux repository, start and enable the service, and grant the ec2-user permission to run Docker commands:

```
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Exit the SSH session and reconnect so the group changes take effect.

Step 4: Create Project and Install Dependencies

```
mkdir node-docker-app && cd node-docker-app
npm init -y
npm install express
npm install --save-dev nodemon
```

Step 5: Create app.js

Create the main Express application file:

```
const express = require("express");
const app = express();
const PORT = 3000;
app.get("/", (req, res) => {
  res.send("Hello from EC2 Docker App");
});
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 6: Update package.json Scripts

Add start and dev scripts to package.json to support both production and development modes:

```
"scripts": {
  "start": "node app.js",
  "dev":   "nodemon app.js"
}
```

Step 7: Create Dockerfile

Create a Dockerfile in the project root to containerize the application:

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "run", "dev"]
```

Step 8: Create .dockerignore

Prevent node_modules and debug logs from being included in the Docker build context:

```
node_modules
npm-debug.log
```

Step 9: Build Docker Image

```
docker build -t node-app .
```

Docker pulls the node:18 base image, copies project files, installs dependencies, and packages everything into a Docker image named node-app.

Step 10: Run Container with Live Reload

```
docker run -d \
  -p 3000:3000 \
  -v $(pwd):/app \
```

```
-v /app/node_modules \
node-app
```

The -v $(pwd):/app flag mounts the EC2 project directory into the container's /app directory. Nodemon monitors changes in this mounted volume and automatically restarts the Express server when any file is saved.

Result

The Node.js Express application was successfully deployed inside a Docker container on the AWS EC2 instance and accessed via the browser. The following screenshots confirm the successful execution of the experiment:

Output 1: Initial Application Response

After running the Docker container, the browser accessed http://13.62.49.231:3000 and displayed the initial response from the Express server:

Figure 2: Browser showing 'Hello from EC2 Docker App' at 13.62.49.231:3000

Output 2: Live Reload Verification

After editing app.js inside the EC2 instance to change the response message and saving the file, the browser was refreshed. The updated message appeared instantly without restarting the container, confirming that live reload via Nodemon is fully functional:

Figure 3: Browser showing 'Live Reload Working' after editing app.js — no container restart required

Table 2: Result Summary

Test Case	Expected	Result
Docker container starts successfully	Running	✓ **PASS**
App accessible at port 3000 via browser	Accessible	✓ **PASS**
Initial message displayed correctly	Shown	✓ **PASS**
File change triggers live reload	Reload	✓ **PASS**
No container restart required	Auto	✓ **PASS**

Conclusion

This case study successfully demonstrated the deployment of a Node.js Express web application inside a Docker container on an AWS EC2 instance running Amazon Linux 2023. The entire workflow — from server setup and application development to containerization and browser access — was performed remotely on the EC2 instance without requiring any local development environment on the developer's machine.

By leveraging Docker volume mounting and Nodemon, a live reload development workflow was achieved where changes made to application source files on the EC2 host are immediately detected and applied inside the container. This mirrors a professional cloud-based DevOps development workflow.

The experiment also highlighted the importance of AWS Security Group configuration for port access and demonstrated the value of Docker containerization in creating consistent, isolated, and reproducible application environments. Future enhancements could include adding Nginx as a reverse proxy, enabling HTTPS using SSL/TLS certificates, integrating a CI/CD pipeline, and deploying the application to a production-ready orchestration platform such as Kubernetes or AWS ECS.

Advantages

- No local setup required — the entire development environment runs on the EC2 cloud instance.
- Docker ensures environment consistency: the application behaves identically across all machines.
- Nodemon provides live reload, significantly accelerating the development and testing cycle.
- Volume mounting enables instant reflection of host file changes inside the running container.
- AWS EC2 offers scalable compute resources that can be easily upgraded as application demand grows.
- Docker isolation prevents dependency conflicts with other software running on the host instance.
- The Dockerfile enables fully reproducible builds that any team member can replicate identically.
- The approach supports collaboration since the EC2 instance can be accessed by multiple developers.

Limitations

- The Docker CE stable repository returned a 404 error during metadata fetch. Docker was installed from the Amazon Linux repository, which may not always provide the latest Docker CE version.
- The application runs over HTTP (not HTTPS), making it unsuitable for production use without SSL/TLS.
- The EC2 public IP address is dynamic and changes on instance restart unless an Elastic IP is allocated.
- Live reload via Docker volume mounting is a development-only approach; production deployments should use built images without host directory mounts.
- Running nodemon directly in CMD is not recommended for production — a process manager such as PM2 should be used instead.
- No reverse proxy (e.g., Nginx) is configured, leaving the Node.js process directly exposed on port 3000 without load balancing or SSL termination.
- The Docker run command does not specify a --restart policy, so the container will not automatically recover if it crashes unexpectedly.

References

[1] Amazon Web Services, "Amazon EC2 Documentation," AWS, 2024. [Online]. Available: https://docs.aws.amazon.com/ec2/

[2] Docker Inc., "Docker Documentation — Get Started," Docker Docs, 2024. [Online]. Available: https://docs.docker.com/get-started/

[3] Node.js Foundation, "Node.js v18 LTS Documentation," Node.js, 2024. [Online]. Available: https://nodejs.org/en/docs/

[4] Express.js Contributors, "Express 5.x API Reference," ExpressJS, 2024. [Online]. Available: https://expressjs.com/en/5x/api.html

[5] Nodemon Authors, "Nodemon — Monitor for any changes in your Node.js application," GitHub, 2024. [Online]. Available: https://github.com/remy/nodemon

[6] AWS Documentation, "Amazon EC2 Security Groups for Linux Instances," AWS, 2024. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html